Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Sketchly: A Browser-Based Collaborative Diagramming Tool with CNN Shape Recognition and Java code Generation

By
Hammad Mubarik

April 26, 2026

B.Sc. (Hons) in Software Development

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

1 Introduction **2**
1.1 Background and Motivation . . . 2
1.2 The Sketchly System . . . 3
1.3 Objectives . . . 4
1.4 Scope and Limitations . . . 4
1.5 Report Structure . . . 5

2 Methodology **6**
2.1 Software Development Methodology . . . 6
2.1.1 Iterative, Milestone-Driven Development . . . 6
2.1.2 Version Control and Branch Strategy . . . 7
2.1.3 Technology Choices and Stack . . . 7
2.2 Research Methodology . . . 8
2.2.1 Approach to Technology Selection . . . 8
2.2.2 Evaluation Approach . . . 9

3 Technology Review **10**
3.1 Existing Diagramming Tools . . . 10
3.1.1 Formal Diagramming Editors . . . 10
3.1.2 Freehand Sketching Tools . . . 10
3.1.3 The Gap . . . 11
3.1.4 The tldraw Canvas Library . . . 11
3.2 Sketch and Gesture Recognition . . . 12
3.2.1 Geometric and Feature Based Approaches . . . 12
3.2.2 Convolutional Neural Networks . . . 12
3.2.3 Recurrent and Sequential Approaches . . . 13
3.2.4 The Quick, Draw! Dataset . . . 13
3.2.5 Browser Based Inference with TensorFlow.js . . . 13
3.3 Real Time Collaborative Editing . . . 14
3.3.1 The Problem of Concurrent Edits . . . 14
3.3.2 Operational Transformation . . . 14
3.3.3 Conflict Free Replicated Data Types . . . 15
3.3.4 Yjs . . . 15
3.4 Graph Layout and Connector Routing . . . 15
3.4.1 The Routing Problem . . . 15
3.4.2 Computational Complexity . . . 16
3.4.3 A* Search . . . 16
3.4.4 Alternative Routing Approaches . . . 16
3.5 UML and Diagram Notation . . . 17
3.6 AI Powered Code Generation . . . 17

3.6.3 Limitations of LLM Based Code Generation 18
3.7 Summary 19

4 System Design **20**
4.1 Overall System Architecture 20
4.2 Freehand Shape Recognition 21
4.2.1 Recognition Pipeline 21
4.2.2 CNN Model Architecture 23
4.2.3 Synthetic Training Data 25
4.3 UML Class Diagram Support 25
4.3.1 Custom tldraw Shape 25
4.3.2 Three-Section Layout and Auto-Resize 26
4.3.3 Connection Points 29
4.4 Connector Routing 29
4.4.1 Grid Representation 29
4.4.2 A* Search Implementation 30
4.4.3 Auto-Snap and Connection Point Detection 31
4.5 Real-Time Collaboration 31
4.5.1 CRDT Data Model 31
4.5.2 WebSocket Synchronisation Server 31
4.5.3 Awareness Protocol 32
4.6 AI-Powered Java Code Generation 32
4.6.1 Canvas Export 32
4.6.2 Backend Endpoint 33
4.6.3 Response Processing and Display 34
4.7 Persistence and Authentication 37
4.7.1 User Authentication 37
4.7.2 Diagram Persistence 37

5 System Evaluation **38**
5.1 Objective 1: Freehand Shape Recognition 38
5.1.1 Evaluation Method 38
5.1.2 Results 38
5.1.3 Analysis 39
5.1.4 Comparison with the Original Geometric Recogniser 39
5.2 Objective 2: Real Time Collaborative Editing 40
5.2.1 Evaluation Method 40
5.2.2 Results 40
5.2.3 Limitations 40
5.3 Objective 3: Intelligent Connector Routing 41
5.3.1 Evaluation Method 41

5.3.2 Results . . . 41
5.3.3 Limitations . . . 41
5.4 Objective 4: UML Class Diagram Support . . . 42
5.4.1 Evaluation Method . . . 42
5.4.2 Results . . . 42
5.5 Objective 5: AI Powered Java Code Generation . . . 42
5.5.1 Evaluation Method . . . 42
5.5.2 Results . . . 43
5.5.3 Limitations . . . 43
5.6 Performance Characteristics . . . 44

6 Conclusion **45**
6.1 Summary . . . 45
6.1.1 Objective 1: Freehand Shape Recognition . . . 45
6.1.2 Objective 2: Real-Time Collaborative Editing . . . 46
6.1.3 Objective 3: Intelligent Connector Routing . . . 46
6.1.4 Objective 4: UML Class Diagram Support . . . 46
6.1.5 Objective 5: AI-Powered Java Code Generation . . . 46
6.2 Reflections on Technical Decisions . . . 47
6.2.1 Synthetic Data Augmentation . . . 47
6.2.2 Hybrid CNN and Geometric Correction . . . 47
6.2.3 CRDTs over Operational Transformation . . . 47
6.2.4 In-Browser CNN Inference . . . 48
6.3 Limitations and Future Work . . . 48
6.3.1 Formal User Study . . . 48
6.3.2 Extended UML Support . . . 48
6.3.3 Fine-Tuning on Real Hand-Drawn Data . . . 48
6.3.4 Offline and Progressive Web App . . . 49
6.3.5 Multi-Language Code Generation . . . 49
6.4 Conclusion . . . 49

A Source Code Repository **53**

List of Figures

4.1 High-level system architecture of Sketchly. 21
4.2 CNN architecture for freehand shape recognition. 24
4.3 Recognised shapes on the Sketchly canvas, with connection points (red dots) showing available anchor locations. 26
4.4 Several recognised shape classes with their connection anchors visible. 27
4.5 Two UML class shapes connected by an orthogonally-routed connector. 30
4.6 Two users collaborating in real time on a shared canvas. 32
4.7 The Java code generation modal showing generated class definitions. 35
4.8 A hand-drawn UML class diagram (left) and the corresponding generated Java code (right). 36

List of Tables

4.1 Technology stack by layer. 22

5.1 Per class precision, recall, and F1 score on the held out test set. . . 39
5.2 Assessment of Java code generation correctness for two test diagrams. 43
5.3 Observed performance characteristics of key system components. . . 44

Chapter 1

Introduction

1.1 Background and Motivation

Diagramming is a fundamental part of software engineering. From sketching a system on a whiteboard in a design meeting to producing formal architecture documentation for a client, engineers rely on diagrams at nearly every stage of development [1]. They are fast to produce, more expressive than code alone, and accessible to stakeholders who may not be fluent in any particular programming language.

Despite this, the tools available for creating software diagrams generally fall into one of two categories, and each has a significant drawback.

The first category is formal diagramming editors: tools like Microsoft Visio, Lucidchart, and draw.io. These offer large shape libraries, support for established notations like the Unified Modelling Language (UML), and clean exportable output. The tradeoff is that they require working within a structured palette and canvas model. Shapes must be selected from a menu, placed precisely, and connected manually. Formatting takes time. That overhead is acceptable when producing final documentation but becomes a real friction during early-stage design, when ideas are still forming and speed matters.

The second category is freehand sketching tools: applications like Excalidraw and digital whiteboards. These are fast and flexible. A circle or an arrow can be drawn in seconds without opening a menu. The problem is that the output is a static image with no underlying structure. There is no way to reuse a hand-drawn class box programmatically, reroute a connector when a shape moves, or export the content in any machine-readable form. The diagram is just pixels.

The gap between these two categories is the problem this project addresses. A tool that interprets freehand strokes and converts them into structured diagram elements would give designers the speed of sketching with the reusability of a

formal editor. Recent advances in browser-based machine learning have made this practical: Convolutional Neural Networks (CNN's) can classify shapes in tens of milliseconds running directly in the browser, with no server round-trip needed per stroke. At the same time, Yjs a JavaScript library can be used for real-time data synchronisation. It has made it straightforward to build multi-user applications that stay consistent without complex conflict resolution logic.

1.2 The Sketchly System

Sketchly is a web-based collaborative diagramming tool built to close this gap. It takes freehand strokes from one or more users, classifies them in real time using a CNN running entirely in the browser, and converts them into clean vector shapes on the canvas. A geometric corner-counting stage always runs after the CNN and can override its result when the physical corner structure of the stroke contradicts the classification.

Ten shape classes are supported: circles, squares, rectangles, triangles, diamonds, directional arrows (left, right, up, down), and straight lines. On top of general shape recognition, Sketchly provides dedicated support for UML class diagram notation. Drawing a rectangle or a square produces a three-compartment UML class box with editable sections for the class name, attributes, and methods. Line connectors between shapes are routed as orthogonal paths using a grid-based A* pathfinding algorithm [2], which keeps connection lines from passing through the bodies of other shapes.

Multiple users can edit the same diagram at the same time. The collaboration layer is built on Conflict-free Replicated Data Types (CRDTs) [3] via Yjs, which guarantees that concurrent edits from any number of users converge to an identical shared state, without needing a centralised server to arbitrate conflicts. Changes propagate through a WebSocket server in under 100ms under normal network conditions.

Once a UML diagram is drawn, users can generate executable Java code from it with a single click. The canvas is rendered as an image and sent to the Claude Opus language model, which reads the visual structure of the diagram and produces Java class definitions that match it, including inheritance hierarchies, field names, and method signatures.

1.3 Objectives

The five objectives of this project are:

1. **Freehand shape recognition.** Design and implement a shape recognition system that converts hand-drawn strokes into clean vector diagram elements in real time, using a CNN trained on synthetic data combined with a geometric corner-counting stage that can override the CNN result when the stroke's physical structure contradicts it.

2. **Real-time collaborative editing.** Support collaborative editing by multiple simultaneous users, where all users see a consistent shared canvas without data loss or conflicts from concurrent modifications.

3. **Intelligent connector routing.** Implement a connector routing system that finds orthogonal paths between diagram elements while navigating around other shapes on the canvas.

4. **UML class diagram support.** Provide dedicated support for UML class diagram notation, including editable three section class boxes and connectors that snap to meaningful anchor points on each shape.

5. **AI-powered Java code generation.** Integrate a pipeline that converts a completed UML class diagram into executable Java source code by sending a rendered image of the diagram to a large language model.

1.4 Scope and Limitations

The project targets the class diagram subset of UML. Of the fourteen diagram types in the UML 2.5.1 specification [4], the class diagram is the most commonly used in practice [1] and the one most often sketched during early-stage object-oriented design. Other diagram types such as sequence diagrams and state machines were out of scope, though the system architecture does not prevent adding further shape types later.

A formal user study was also out of scope. Recognition accuracy and collaborative consistency were assessed through empirical testing, and the limitations of this approach are discussed in the conclusion.

1.5 Report Structure

Chapter 2 describes the development and research methodology. Chapter 3 reviews existing tools and relevant literature across the five technical areas. Chapter 4 covers the system design and implementation in detail. Chapter 5 evaluates the system against the objectives above. Chapter 6 summarises findings and outlines directions for future work.

Chapter 2

Methodology

This chapter covers how Sketchly was built and how design decisions were made. The two are closely related but separate: the development methodology is about how the software was planned and iterated on, while the research methodology is about how the right technical approach was chosen for each component.

2.1 Software Development Methodology

2.1.1 Iterative, Milestone-Driven Development

The project was developed in five milestones, each corresponding to one of the system's five objectives:

1. Freehand shape recognition and stroke-to-shape conversion
2. Real-time collaborative editing
3. Intelligent orthogonal connector routing
4. UML class diagram support with editable compartments
5. AI-powered Java code generation from UML diagrams

Each milestone was scoped, built, and integrated before moving to the next, so the application stayed in a working state throughout development. At any point during the project, a browser could be opened, shapes drawn on the canvas, and the result seen immediately, even if later features were not yet present.

This approach lines up with agile software development principles [5], which favour working software delivered in increments over designing everything upfront. In practice, working iteratively served two purposes here. First, it meant technical

uncertainties could be surfaced and resolved early. The clearest example was shape recognition: an initial geometric recogniser was built first, but it quickly became clear through testing that it misclassified shapes drawn quickly, at odd angles, or with the kind of hand tremor that is normal when sketching freehand. That led to replacing it with a CNN as the primary recogniser, keeping the geometric approach only as a post-processing correction step. Because this happened at the end of the first milestone, it did not block work on collaboration, routing, or UML support.

Second, building features in sequence meant each later feature could be designed with real knowledge of what came before. The connector routing system was built knowing the exact shape of the connection point data model, and the UML class shape was designed knowing exactly how tldraw's shape extension API worked in practice.

2.1.2 Version Control and Branch Strategy

Git was used for version control throughout the project. All development happened on the `main` branch. Commits were made frequently, typically after each meaningful addition or bug fix, to keep a clear record of progress and make it easy to recover earlier working states when regressions appeared.

Working directly on `main` rather than using feature branches was a deliberate choice for a solo project. Feature branches are most valuable in team settings where parallel workstreams need to be kept separate. For a single developer working in short focused sessions, committing directly to `main` with small, frequent commits produced a cleaner and more useful history than branching would have.

2.1.3 Technology Choices and Stack

The core technology stack was fixed at the start of the project and stayed consistent throughout. The frontend was built with React and TypeScript, using the tldraw library [6] as the canvas layer. tldraw handles pointer events, coordinate transforms, zooming, panning, shape rendering, and undo/redo history, all of which would have taken substantial time to build from scratch. Starting from tldraw freed development effort for the parts that were the actual contribution of the project: shape recognition, routing, collaboration, and UML support.

Real-time collaboration runs on Yjs [7], a CRDT library for JavaScript. Yjs was chosen early on because it ships a binding for tldraw, meaning any canvas change is automatically propagated to all connected clients without writing any conflict resolution logic. It communicates over a lightweight WebSocket server (`y-websocket`) running on Node.js. The decision to use Yjs rather than building a custom synchronisation layer saved a significant amount of time. The backend

uses Express.js and Node.js, which share the same language as the frontend (TypeScript). The machine learning training pipeline runs in Python using TensorFlow and Keras, which support exporting trained models directly to TensorFlow.js format for browser deployment.

2.2 Research Methodology

2.2.1 Approach to Technology Selection

For each of the five components, the selection process was the same: review what approaches already exist, compare them against the specific requirements of this project, pick the most appropriate one, and build it. The full reasoning for each choice is in the Technology Review chapter. A short summary follows here.

Shape recognition: The literature was reviewed for sketch and gesture recognition approaches. Two main families were found: geometric and feature-based classifiers, and neural network classifiers. The $1 Recogniser [8] was the most popular approach and was implemented first. When it proved to be unreliable for freehand input, CNN-based recognition was adopted instead.

Training data: No suitable public dataset of hand-drawn software diagram shapes exists. The Quick, Draw dataset [9] covers everyday objects but not diagram-specific shapes like connector arrows or UML class boxes. A synthetic dataset was generated from scratch, with augmentation steps designed to replicate realistic freehand variation.

Collaboration: The literature on distributed consistency was reviewed, focusing on Operational Transformation [10, 11] and CRDTs [3]. Yjs [7] was chosen as the CRDT implementation because of how reliable it is, its existing tldraw integration, and its suitability for multi-user and peer to peer scenarios.

Connector routing: A* [2] was chosen over Dijkstra's algorithm and visibility graph routing because it finds optimal orthogonal paths on a grid using an admissible heuristic, and orthogonal routing is the standard expectation in UML and software architecture diagrams.

Code generation: Claude Opus [12] was selected after reviewing the literature on LLM code generation capability [13]. It can process an image of a diagram and produce corresponding source code, which is a vision-language task that requires understanding both the diagram structure and the semantics of the target language.

2.2.2 Evaluation Approach

Each component was evaluated in a way suited to its nature:

- **Shape recognition** was measured quantitatively with a synthetic test set, reporting per-class precision, recall, and F1 score. The overall test accuracy was 99.28%, with full per-class results in Chapter 5.
- **Collaborative editing** was tested by opening the same room in multiple different browsers, for example User A in Google Chrome and User B in Firefox, and making concurrent edits, checking that all changes propagated without loss or duplication and that all clients ended up with an identical canvas.
- **Connector routing** was evaluated visually by building diagrams with overlapping shapes and confirming that all routed paths stayed outside shape boundaries.
- **UML class shapes** were tested by typing and pasting text into all three compartments, checking auto-resize behaviour, and verifying that cursor position survived each re-render.
- **Java code generation** was assessed by drawing a UML diagram adding fields, methods and other attributes and manually checking the output for correctness of class structure, inheritance, field declarations, and method signatures.

Chapter 3

Technology Review

This chapter reviews existing tools and literature relevant to the five objectives of the project: freehand shape recognition, real time collaborative editing, intelligent connector routing, UML class diagram support, and AI powered code generation. For each area, prior work is examined, alternatives are compared, and the reasoning behind the choices made in Sketchly is explained.

3.1 Existing Diagramming Tools

3.1.1 Formal Diagramming Editors

Microsoft Visio and Lucidchart are among the most widely used diagramming tools in professional settings. Both provide large shape libraries, support established notations including UML, and produce clean, formatted output. They support real time collaboration (Lucidchart natively, Visio through Microsoft 365 integration) and export to a range of formats. The tradeoff is that both require working through a palette and canvas model: shapes are selected from a menu, dragged onto the canvas, and connected by drawing from anchor point to anchor point. This is fine for producing polished documentation but too slow for exploratory design, where the goal is to get ideas down quickly rather than format them precisely.

draw.io follows a similar model but is free and runs entirely in the browser. It is widely used in both academic and professional settings and supports a wide range of shape libraries. Like Visio and Lucidchart, it requires structured input and has no mechanism for interpreting freehand strokes.

3.1.2 Freehand Sketching Tools

Excalidraw sits at the other end of the spectrum. It is a web based sketching tool that renders shapes with a hand drawn aesthetic. Users draw by clicking

and dragging, and the output is styled to look informal and sketch like. It is fast, supports collaboration, and is popular for brainstorming and informal communication. However, it does not interpret freehand strokes as semantic diagram elements. There is no shape recognition, and the content of an Excalidraw diagram cannot be exported as structured data. A rectangle drawn in Excalidraw is just a styled polygon, not a class or component with attached metadata.

3.1.3 The Gap

None of the tools above combine freehand stroke input, automatic semantic conversion to structured diagram elements, real time multi user collaboration, and AI powered code generation in one application. Sketchly was built to fill that gap.

Four distinct technologies make this possible. A CNN running via TensorFlow.js [14] classifies hand drawn strokes into ten shape categories directly in the browser, with no server round trip needed. Yjs [7], a CRDT library for JavaScript, provides real time collaborative synchronisation so multiple users can edit the same diagram concurrently without any centralised conflict resolution. A grid based A* pathfinder [2] routes connectors along orthogonal paths that navigate around other shapes on the canvas. And Claude Opus [12], a vision language model, reads a rendered image of the completed diagram and generates executable Java source code from it.

Each of these four components fills a gap that no single existing tool addresses. The sections below look at each area in depth, comparing the chosen approach against alternatives.

3.1.4 The tldraw Canvas Library

Rather than building a canvas from scratch, Sketchly is built on tldraw [6]. tldraw is an open source JavaScript library providing an infinite canvas with pointer event handling, coordinate transforms, zoom and pan, SVG shape rendering, undo/redo history, and a pluggable shape system that lets developers define custom shape types alongside the built in ones.

Building on tldraw rather than from scratch is a straightforward engineering decision: it removes a large category of low level bugs in pointer handling and coordinate arithmetic, and focuses development effort on the parts of the project that are the actual contribution. The tldraw shape extension API made it possible to add the UML class shape as a first class canvas citizen without touching the library's own code.

3.2 Sketch and Gesture Recognition

3.2.1 Geometric and Feature Based Approaches

Early gesture recognition work focused on extracting hand crafted features from stroke paths. Rubine [15] proposed a linear classifier trained on features like the initial stroke angle, bounding box aspect ratio, total stroke length, and maximum deviation from a straight line between the start and end points. It is computationally cheap and works well when the gesture vocabulary is small and classes are well separated in feature space, but accuracy degrades as the vocabulary grows or as input becomes noisier. Rubine's classifier was designed for a small set of command gestures, not the broader range of shapes that appear in software diagrams.

The $1 Recogniser by Wobbrock et al. [8] takes a different approach that needs no training phase. Strokes are resampled to a fixed number of evenly spaced points, rotated to a canonical orientation, scaled to fit a reference square, and then matched against stored templates using a point to point distance measure. Classification is nearest neighbour over the template set. Since templates are just recorded examples of each gesture class, no labelled dataset is needed and the approach is straightforward to implement.

The $1 Recogniser was the initial recogniser in Sketchly. It did not hold up well in the diagramming context. Performance dropped on strokes drawn quickly, with hand tremor, at unusual scales, or with partial incompleteness such as a circle that is not quite closed. It is also sensitive to drawing direction: a square drawn from the top left corner clockwise and a square drawn from the bottom right corner counter clockwise produce different point sequences, and the nearest neighbour search may not identify them as the same shape without templates for every valid drawing order.

3.2.2 Convolutional Neural Networks

CNNs became dominant for image classification after AlexNet's result at the 2012 ImageNet challenge [16], which showed that deep CNNs trained on large labelled datasets could substantially outperform hand crafted feature engineering on complex classification tasks.

The theoretical foundations go back further. LeCun et al. [17] showed with LeNet that CNNs could learn spatial hierarchies of features directly from raw pixel data, from edges through to shapes, without any manual feature extraction. That property makes CNNs a natural fit for freehand sketch recognition: the relevant features of a drawn shape (the angles of its sides, the curvature of its outline, the arrangement of its corners) are learned from examples rather than coded by hand.

For Sketchly, choosing a CNN over a geometric classifier came down to robustness. A CNN operating on a rasterised 48×48 image of the completed stroke is insensitive to drawing order and direction, handles scale variation because the image is normalised before inference, and can learn the subtle visual features that separate ambiguous shape classes. The cost is needing a labelled training dataset and a training pipeline, but since the data could be generated synthetically (as described in Chapter 2), that cost was manageable.

3.2.3 Recurrent and Sequential Approaches

Ha and Eck [18] proposed Sketch RNN, a generative recurrent model that treats a sketch as a sequence of pen strokes. The model encodes the temporal sequence of drawing actions and can both generate and classify sketches at the stroke level, preserving information about drawing order and speed that a static image classifier throws away.

Sketch RNN is considerably more complex to train and deploy than a CNN image classifier, and its advantages show most clearly for fine grained discrimination between visually similar sketches where drawing order carries meaningful signal. For the task of distinguishing between ten diagram shape categories, circles, squares, rectangles, triangles, and so on, the temporal information adds little, and a CNN on the rasterised stroke is both sufficient and much simpler.

3.2.4 The Quick, Draw! Dataset

The Google Quick, Draw! dataset [9] contains tens of millions of labelled hand drawn sketches across 345 categories, collected through a web based game where participants draw named objects in under 20 seconds. It has been widely used as training and evaluation data for sketch recognition models.

It is not suitable for training a diagramming shape recogniser for two reasons. First, it does not include diagram specific shape classes like connector arrows in multiple directions, UML style rectangles with specific aspect ratios, or diamonds with vertex aligned geometry. Second, the drawing style of Quick, Draw! contributors, who draw named objects quickly against a timer, is different from the style of someone sketching a software diagram deliberately. A diagramming recogniser needs to generalise to slower, more controlled drawing, which is not well represented in Quick, Draw!

3.2.5 Browser Based Inference with TensorFlow.js

Running neural network inference as part of a client side web application requires a runtime that works in the browser. TensorFlow.js [14] does this by compiling

model computations to WebGL shader programs that execute on the device GPU. Recognition completes in the browser without a server round trip, which matters a lot for a tool where shape classification needs to feel instantaneous. Waiting several hundred milliseconds for a server response after every stroke would break the freehand experience.

The alternative of sending stroke data to a backend server for classification introduces network latency that varies and cannot be controlled, requires the server to scale with the number of active users, creates a hard dependency on network availability, and sends user diagram content to a remote server. Running inference in the browser avoids all of that, at the cost of a slightly larger initial page load since the model weights (around 400KB) are served as a static asset.

3.3 Real Time Collaborative Editing

3.3.1 The Problem of Concurrent Edits

When multiple users edit a shared document at the same time, each client must apply incoming changes from other users against whatever local changes have not yet been sent. The challenge is that changes arrive from multiple clients with no shared global ordering, and naive approaches like last write wins can easily lead to lost work or awkward states of shapes.

3.3.2 Operational Transformation

Operational Transformation (OT), introduced by Ellis and Gibbs [10], was the first widely adopted approach to this problem in collaborative text editing. OT works by transforming incoming operations against any operations applied locally since the last common ancestor state. The transformation function is designed so that applying a transformed operation produces the same result regardless of the original arrival order. All clients eventually reach the same state.

Google Docs uses a variant of OT [11], and it works well at scale for text editing. The well known limitation of OT is that correctness depends on all operations passing through a centralised server that imposes a total ordering. Without that server, the transformation function can produce wrong results when three or more concurrent operations interact in ways the binary transformation does not account for [19]. This makes OT difficult to implement correctly in peer to peer or multi server settings, and requires maintaining a persistent server even for small sessions.

3.3.3 Conflict Free Replicated Data Types

CRDTs, formalised by Shapiro et al. [3], take a different approach. A CRDT is a data structure whose merge operation is commutative, associative, and idempotent: any two replicas can be merged in any order and always produce the same result. Eventual consistency is guaranteed without needing a central server to impose an ordering.

Two families of CRDTs exist. State based CRDTs (CvRDTs) periodically exchange full state replicas and merge them. Operation based CRDTs (CmRDTs) broadcast individual operations, and the data structure is designed so that any ordering of operations converges [3]. For collaborative editing, operation based CRDTs are generally preferred because broadcasting one operation is far more bandwidth efficient than broadcasting the entire document state.

3.3.4 Yjs

Yjs [7] is a production ready, operation based CRDT library for JavaScript, providing shared data types including `Y.Array`, `Y.Map`, and `Y.Text`. It uses the YATA (Yet Another Transformation Approach) algorithm to resolve concurrent insertions at the same position by assigning a total order based on unique client identifiers, avoiding the interleaving problems that affected earlier CRDT designs.

Yjs benchmarks well against alternatives like Automerge [20] in terms of memory efficiency and operation throughput, and is used in production by a number of collaborative tools. For this project, the key factor was that a Yjs binding for tldraw already exists. Any change made by one user, adding a shape, moving it, editing its text, is automatically written to the shared Yjs document and propagated to every other connected client without any additional application level code.

Kleppmann et al. [20] make the case for local first software, where the client holds a full local replica and can work offline, syncing when connectivity returns. Yjs supports this model through its CRDT properties, which makes it a good fit for a diagramming tool where users might want to keep working through a network interruption.

3.4 Graph Layout and Connector Routing

3.4.1 The Routing Problem

In a diagramming tool, connectors between elements should follow paths that are visually clear. In UML and software architecture diagrams, the convention is orthogonal (rectilinear) paths that turn only at right angles. A connector passing

through the body of another shape is visually confusing and implies a relationship that does not exist. The routing problem is to find an orthogonal path from a source connection point to a target that avoids passing through other shapes.

3.4.2 Computational Complexity

The general problem of drawing a graph with minimum crossings is NP hard [21], which rules out exact polynomial time solutions for large graphs. In practice, diagramming tools work with small numbers of shapes, typically tens rather than thousands, shapes that are mostly axis aligned rectangles, and connectors expected to follow orthogonal paths. These constraints make efficient approximations tractable.

3.4.3 A* Search

A* [2] is a best first search that finds the shortest path in a weighted graph using a heuristic to guide the search toward the goal. It guarantees an optimal result when the heuristic is admissible, meaning it never overestimates the true remaining cost.

For connector routing in Sketchly, the canvas is divided into a uniform grid of 25 pixel cells. Shape bounding boxes are marked as impassable obstacles. A* then finds the shortest orthogonal path from the source connection point to the target, moving through unblocked cells. The Manhattan distance from the current cell to the goal is the heuristic. Since the grid allows only horizontal and vertical moves, Manhattan distance is admissible: it equals the true shortest path cost when no obstacles are present and never overestimates when they are. A* is therefore guaranteed to find the shortest obstacle free orthogonal path.

3.4.4 Alternative Routing Approaches

Dijkstra's algorithm is mathematically equivalent to A* with a zero heuristic. It explores cells in order of distance from the source without any guidance toward the goal, so it is slower in practice. For typical diagram sizes the difference is small, but A* is used in Sketchly for better performance on large canvases.

Visibility graph routing finds shortest paths by considering only the corners of obstacles [22]. This can produce shorter paths than grid based A* since it is not limited to a fixed grid resolution, but visibility graph paths naturally include diagonal segments. Orthogonalising them is non trivial, and the approach was not pursued.

Straight line routing simply draws a direct line between the two connection points. Some tools use it for performance reasons, but it is not acceptable here

because straight line connectors pass through shapes that lie between source and target, which is visually confusing in any dense diagram.

3.5 UML and Diagram Notation

The Unified Modelling Language is a standardised notation for describing software systems maintained by the Object Management Group [4]. Version 2.5.1 defines fourteen diagram types: seven structural (class, object, component, package, composite structure, profile, and deployment) and seven behavioural (use case, activity, state machine, sequence, communication, interaction overview, and timing).

The class diagram is the most widely used of the fourteen [1]. It represents the static structure of a system: the classes, their attributes and methods, and the relationships between them (inheritance, association, dependency, composition, aggregation). It is also the diagram type most commonly sketched during early stage object oriented design, which makes it the natural focus for a freehand diagramming tool.

A UML class is drawn as a rectangle divided into three horizontal compartments: the top holds the class name, the middle lists attributes with their types, and the bottom lists methods with their signatures. Relationships are shown as lines with specific arrowhead and line style conventions: a solid line with a hollow triangular arrowhead for inheritance, a dashed line for dependency, a solid diamond for composition, and so on.

Sketchly supports UML class shapes with editable compartments and recognises drawn arrows as connectors. Full coverage of all fourteen diagram types and all UML relationship styles is beyond scope, but the three compartment class box with cardinal connection points covers the core of what UML class diagrams need.

3.6 AI Powered Code Generation

3.6.1 Large Language Models for Code

LLM based code generation has advanced rapidly. Chen et al. [13] evaluated Codex, a GPT-3 based model fine tuned on public GitHub code, on the HumanEval benchmark, a set of 164 Python problems specified in natural language. Codex solved 28.8% of problems with a single attempt, and substantially more when multiple samples were generated and the best selected. This showed that LLMs could produce syntactically correct and functionally useful code from informal descriptions.

Subsequent models, including Anthropic's Claude family [12], have pushed this further. Modern LLMs can generate code from structured inputs like class names,

method signatures, and relationship descriptions. Output quality is sensitive to input clarity: well structured, unambiguous inputs tend to produce correct code.

3.6.2 Vision Language Models for Diagram to Code

Generating code from a visual diagram rather than a text description adds the challenge of diagram interpretation. The model must identify the classes in the image, read their names and compartment contents, understand which arrows represent which relationships, and then generate corresponding source code.

This is a vision language task requiring a multimodal model. Claude Opus [12] accepts an image as part of its input and can describe, reason about, and generate code from what it sees. For UML to Java generation, the instruction is simple: given a UML class diagram image, produce Java class definitions matching the structure shown, using the arrow types to determine inheritance and implementation relationships.

3.6.3 Limitations of LLM Based Code Generation

The main limitation is dependence on the quality of the input diagram. A clearly drawn, unambiguous diagram consistently produces correct code. A diagram where shapes overlap, labels are too small to read, or arrow directions are unclear will likely produce incorrect or incomplete output. The model may also add method bodies or field types that were never in the diagram, drawing on its training data to fill gaps. These issues are discussed further in the evaluation chapter.

3.7 Summary

The paper points to a clear gap in existing tools. Formal editors like Visio, draw.io, and Lucidchart produce high quality structured output but need structured input and are slow for exploratory design. Freehand tools like Excalidraw are quick and natural but produce nothing reusable. No existing tool brings together freehand input, shape recognition, orthogonal connector routing, real time multi user collaboration, and AI code generation.

The technical choices in Sketchly are each backed by the literature as the most appropriate for the specific requirements: a CNN for robust sketch classification, Yjs CRDTs for collaborative consistency without a centralised arbiter, A* for optimal orthogonal routing, and a vision language model for semantically aware diagram to code generation.

Chapter 4

System Design

This chapter covers the design and implementation of Sketchly, starting with the overall architecture and then working through each major component: freehand shape recognition, UML class diagram support, connector routing, real-time collaboration, AI-powered code generation, and persistence.

4.1 Overall System Architecture

Sketchly is a three-tier web application with a clear split of responsibilities between the frontend client, the backend server, and the machine learning training pipeline.

Frontend (React + TypeScript). The frontend is a single page React application built with TypeScript, bundled with Vite. This is where the user interacts with the system: drawing strokes, editing text in UML shapes, viewing routed connectors, and triggering code generation. All shape recognition inference runs inside the browser using TensorFlow.js, so there is no server round-trip per stroke. The frontend talks to the backend over two channels on the same origin: HTTP REST requests for Java code generation, and a WebSocket upgrade on the same port for real-time collaborative synchronisation via Yjs. During local development the WebSocket service can also be run as a separate process on port 1234.

Backend (Express.js + Node.js). The backend does two things in a single Node process. It exposes a REST endpoint (`POST /api/generate-java`) that accepts a base64-encoded PNG of the canvas, forwards it to the Claude API, and returns the generated Java source code. It also accepts WebSocket upgrades on the same HTTP server and hands them to the Yjs synchronisation layer (`y-websocket`), which propagates CRDT operations between connected clients and holds the active document state in memory.

Training Pipeline (Python + TensorFlow). The training pipeline is a

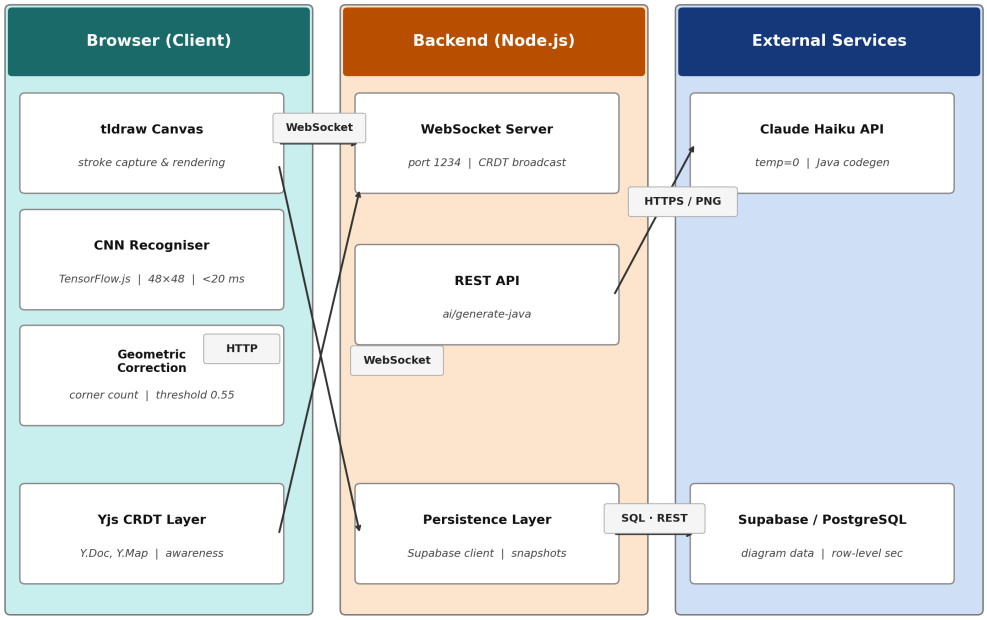

Figure 4.1: High-level system architecture of Sketchly.

standalone Python script that generates synthetic training data, trains the CNN, and exports the weights in TensorFlow.js format. It runs offline on a development machine and is not part of the deployed application. Its output, a `model.json` metadata file and a binary weights file, is written into the frontend's `public/` folder so that it ships with the built bundle and is served as a static asset.

Persistence (Supabase). User accounts and saved diagrams are stored in Supabase, a managed PostgreSQL service with a JavaScript client library. The frontend writes diagram snapshots directly to Supabase, bypassing the Express backend, triggered by an auto-save handler that debounces writes to once every five seconds.

The technology choices for each layer are summarised in Table 4.1.

4.2 Freehand Shape Recognition

4.2.1 Recognition Pipeline

The shape recognition pipeline runs entirely in the browser and fires at the moment the user lifts the pen or releases the mouse button. It has four stages.

Stage 1: Stroke capture. As the user draws, tldraw records a sequence of pointer-move events. Each event contributes an (x, y) coordinate in canvas space.

Layer	Technology	Version / Notes
Frontend framework	React + TypeScript	React 19, TS 5.9
Canvas library	tldraw	v4.1
ML runtime	TensorFlow.js	v4.22
Collaboration	Yjs + y-websocket	Yjs 13.6
Build tool	Vite	v7.1
Backend framework	Express.js	v5.1
AI API client	Anthropic SDK	v0.84
Database / Auth	Supabase	v2.39
ML training	TensorFlow / Keras	2.15 (Python)

Table 4.1: Technology stack by layer.

These build up into an ordered list representing the complete stroke path.

Stage 2: Rasterisation and normalisation. The stroke is normalised to fit a 48×48 grayscale image with 10% padding on each side so the shape does not touch the image boundary. The line between every consecutive pair of points is rasterised directly into a 48×48 `Float32Array` using a manual circle-stamp routine at a stroke width of 1.5 pixels, which is narrow enough to represent thin shapes like lines clearly while not dominating the enclosed area of broader shapes. The resulting pixel array is normalised to $[0, 1]$ and reshaped into the input tensor.

Stage 3: CNN inference. The normalised tensor is passed to the loaded TensorFlow.js model, which produces a 10 dimensional output vector where each value is the model's confidence that the input belongs to one of the ten shape classes. The highest confidence class is taken as the primary classification. The acceptance threshold is 0.55 for closed shapes and arrows; the *line* class uses a lower 0.25 threshold because lines drawn between shapes are often curved or imprecise. If the confidence falls below the relevant threshold, the stroke is classified as unknown and no shape is created.

Stage 4: Geometric post-processing. Regardless of the CNN's confidence, a geometric corner-counting analysis runs on the original stroke coordinates. The stroke is subsampled to around 40 evenly-spaced points, and the direction change at each point is computed as the angle between the vector from the preceding window and the vector to the following window. A direction change above 0.28π radians (roughly 50 degrees) that occurs far enough from the previous detected corner is counted as a corner. Based on the total corner count:

- Four or more corners: the shape is reclassified as a rectangle or square (distinguished by bounding-box aspect ratio) if the CNN confidence is below 0.78, or if the CNN returned *triangle* or *unknown* at any confidence level.

- Exactly three corners: if the CNN returned `unknown`, the shape is reclassified as a triangle by default, or as a rectangle or square when the stroke nearly closes and has a roughly square aspect ratio (a four-sided box that happened to register only three direction changes). If the CNN returned a low-confidence rectangle or square, the geometric agreement is used to lift the result over the acceptance threshold rather than to rename it.
- Two or fewer corners: if the CNN returned a rectangle or square with confidence below 0.78 *and* the stroke does not nearly close, the shape is reclassified as a triangle, on the assumption that an open U-shape or L-shape was drawn instead of a closed box.

This hybrid approach outperforms either method alone. The CNN handles the vast majority of inputs correctly at high confidence. The geometric correction handles the specific case where the CNN result contradicts the physical corner structure of the stroke, for example a triangle drawn with one very long side that rasterises in a way that looks rectangular. Crucially, the geometric stage can override even a confident CNN result when the corner count is incompatible with that prediction.

4.2.2 CNN Model Architecture

The CNN is a sequential model with three convolutional blocks followed by global average pooling and two fully connected layers. The architecture is shown in Figure 4.2.

Each convolutional block has a `Conv2D` layer with 3×3 kernels and same-padding (preserving spatial dimensions), followed by Batch Normalisation [23] and a ReLU activation. Batch normalisation accelerates training by reducing internal covariate shift between layers, and also acts as a regulariser that reduces overfitting. The three blocks use 32, 64, and 128 filters respectively, progressively increasing the channel depth as spatial resolution halves through each `MaxPooling2D(2×2)` layer.

After the three convolutional blocks, a `GlobalAveragePooling2D` layer computes the average value of each feature map over its spatial extent, producing a 128-dimensional feature vector. This replaces the more common `Flatten` followed by a large dense layer. Global average pooling, introduced by Lin et al. [24], reduces the parameter count in the classifier head and provides a more direct correspondence between feature maps and class confidence. Two dropout layers [25] at rates 0.4 and 0.3 are applied before and after the intermediate dense layer to reduce overfitting further.

The output layer is a 10-neuron dense layer with softmax activation, producing a probability distribution over the ten shape classes.

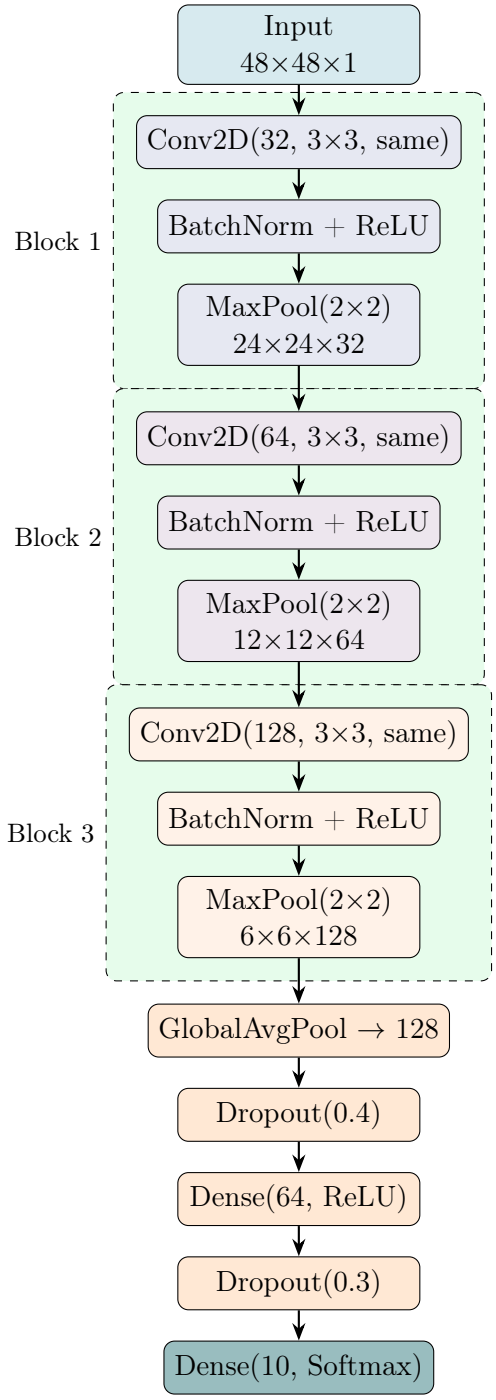

Figure 4.2: CNN architecture for freehand shape recognition.

4.2.3 Synthetic Training Data

The model was trained on 100,000 images (10,000 per class), generated by a Python script. Each image was produced as follows:

1. A geometrically ideal version of the target shape is drawn on a 48×48 blank canvas with a random scale within class specific ranges and a random rotation within class-specific angular limits.

2. **Hand tremor simulation.** Two superimposed sine waves with random frequencies and amplitudes are added to both the x and y coordinates of the stroke path, simulating the oscillatory motion of a hand drawing quickly. Gaussian noise is also added to each point for high-frequency perturbation.

3. **Elastic deformation.** A smoothly varying displacement field is applied to the stroke path, bending and stretching the shape globally to simulate the non-uniform pressure and speed of freehand drawing.

4. **Variable point density.** The density of points along the stroke is varied according to a simulated speed profile based on local curvature. Straight segments have fewer, more widely spaced points (fast drawing); curved segments have more closely spaced points (slower drawing). This reflects the natural tendency to slow down around corners.

5. **Stroke truncation.** For closed shapes (circles, squares, rectangles, triangles, diamonds), the stroke is randomly truncated with probability 0.4, removing a portion of the end path. This simulates the common freehand habit of not quite closing a shape.

6. The augmented stroke is rasterised to the 48×48 canvas using the same pipeline used during inference, so the training and test distributions are matched.

The dataset was split 90/10 into training and sets. Training used a batch size of 64 and ran for up to 100 epochs with early stopping based on validation accuracy (patience of 15 epochs). The best checkpoint by validation accuracy was saved and exported to TensorFlow.js format.

4.3 UML Class Diagram Support

4.3.1 Custom tldraw Shape

The UML class box is a custom tldraw shape type named `uml-class`. tldraw's shape extension API lets developers register custom shape types alongside the

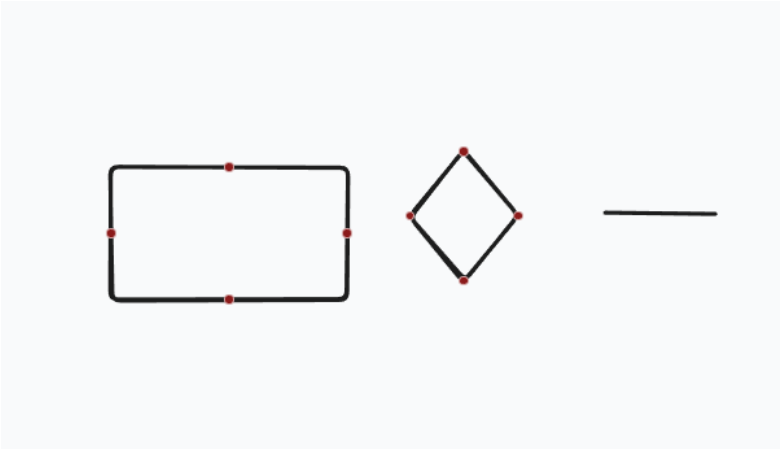

Figure 4.3: Recognised shapes on the Sketchly canvas, with connection points (red dots) showing available anchor locations.

built in ones. Each custom shape type needs:

- A props schema defining the shape's data fields, validated at runtime
- A React component that renders the shape on the canvas
- Geometry definitions used for hit-testing and selection
- Connection point definitions specifying where arrows can attach

The `uml-class` shape stores these props:

Listing 4.1: UML class shape props definition (TypeScript).

```
type UMLClassShape = TLBaseShape<'uml-class', {
  w: number           // width in canvas units
  h: number           // height (auto-managed)
  topText: string     // class name compartment
  middleText: string // attributes compartment
  bottomText: string // methods compartment
}>
```

4.3.2 Three-Section Layout and Auto-Resize

The shape renders as a fixed-width rectangle divided into three horizontal sections by two horizontal dividing lines. Each section contains a `textarea` whose height adjusts to its content. The auto-resize behaviour is implemented with React's `useEffect` hook:

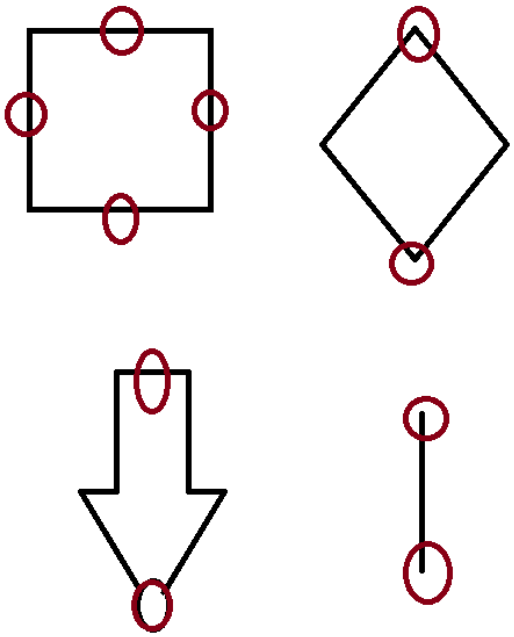

Figure 4.4: Several recognised shape classes with their connection anchors visible.

Listing 4.2: Auto-resize logic triggered on text changes.

```
useEffect(() => {
  if (!topRef.current || !midRef.current || !botRef.current)
     return
  topRef.current.style.height = 'auto'
  midRef.current.style.height  = 'auto'
  botRef.current.style.height  = 'auto'
  const topH = Math.max(MIN_SECTION_H, topRef.current.
     scrollHeight)
  const midH = Math.max(MIN_SECTION_H, midRef.current.
     scrollHeight)
  const botH = Math.max(MIN_SECTION_H, botRef.current.
     scrollHeight)
  topRef.current.style.height = topH + 'px'
  midRef.current.style.height = midH + 'px'
  botRef.current.style.height = botH + 'px'
  const totalH = topH + midH + botH
  if (Math.abs(totalH - shape.props.h) > 1) {
    editor.updateShape({ id: shape.id, type: 'uml-class',
      props: { ...shape.props, h: totalH } })
  }
}, [shape.props.topText, shape.props.middleText, shape.props.
   bottomText])
```

The effect depends only on the three text prop values, not on the shape height. That is intentional: if height were in the dependency array, updating the height would re-trigger the effect, which would update the height again, and so on indefinitely. Depending only on text values means the effect fires once when text changes and then stops.

The second challenge is cursor position. React's controlled component model sets the `textarea` value from state on every render, which causes the browser to reset the cursor to the end of the text after each re-render. Typing a character in the middle of a word would cause the cursor to jump to the end, making normal editing impossible.

The fix is to save the cursor position as a `{start, end}` object in a ref before each `onChange` event, then restore it via `setSelectionRange` inside a `useLayoutEffect` that runs synchronously before the browser paints:

Listing 4.3: Cursor restoration using useLayoutEffect.

```
useLayoutEffect(() => {
  if (topCursor.current && topRef.current) {
    topRef.current.setSelectionRange(
      topCursor.current.start, topCursor.current.end)
    topCursor.current = null
  }
  // repeated for mid and bot sections
})
```

`useLayoutEffect` runs after the DOM is updated but before the browser paints the frame, which makes it the right place for DOM operations that must not be visible as intermediate states.

4.3.3 Connection Points

Each UML class shape exposes four connection points at the cardinal midpoints of its bounding rectangle: top-centre, bottom-centre, left-centre, and right-centre. They are defined in Sketchly's connection-points module and surfaced visually by an overlay component that draws a small dot at each anchor; arrows snapped to a connection point use tldraw's own binding API to attach to the underlying shape. The horizontal dividing lines between compartments are rendered as CSS `border-bottom` rules on the `textarea` elements inside the React component, not as separate tldraw shapes.

4.4 Connector Routing

4.4.1 Grid Representation

The connector routing system is a grid-based A* pathfinder operating in canvas coordinate space. The canvas is divided into 25×25 pixel cells. When a new connector is requested, the routing algorithm builds a grid covering the path's bounding box with additional padding to allow travel around obstacles.

Each shape on the canvas contributes a set of impassable cells. The cells covered by a shape's bounding box (plus a small padding) are marked as blocked, preventing the path from passing through the shape's interior. The source and target connection points need special handling because they sit on the boundary of a blocked rectangle: the algorithm explicitly unblocks the cell at each endpoint and punches a short corridor of cells outward in the connection point's outward normal direction, so the path has a doorway to exit the source shape and enter the target shape without having to walk along the inside of either bounding box.

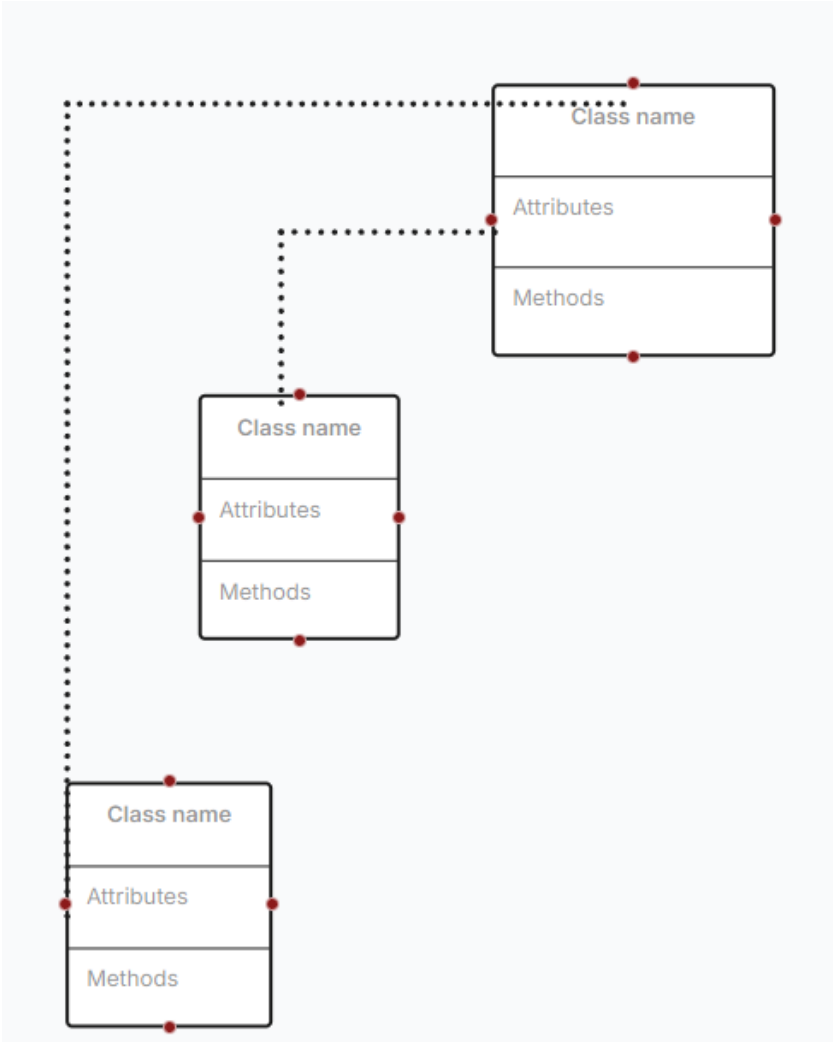

Figure 4.5: Two UML class shapes connected by an orthogonally-routed connector.

4.4.2 A* Search Implementation

The search maintains an open set (a priority queue ordered by $f = g + h$, where g is cost from the start and h is Manhattan distance to the goal) and a closed set of already-evaluated cells. At each step, the lowest-f cell is dequeued, its neighbours in the four cardinal directions are checked, and any unvisited unblocked neighbours are added to the open set with updated costs.

The Manhattan distance heuristic is:

$$h(n) = |n_x - \text{goal}_x| + |n_y - \text{goal}_y|$$

Since the grid allows only horizontal and vertical moves with uniform cost, this is

admissible, so A* is guaranteed to find the shortest obstacle-free orthogonal path [2].

4.4.3 Auto-Snap and Connection Point Detection

When the user releases a connector endpoint near a shape, the system checks whether the endpoint is within 80 pixels (adjusted for the current zoom level) of any registered connection point. If one is found within that threshold, the connector snaps to it. The 80-pixel threshold was chosen through experimentation as the distance at which snapping feels natural: close enough to activate reliably when intended, but far enough not to fire when the user is placing a point nearby without meaning to snap.

4.5 Real-Time Collaboration

4.5.1 CRDT Data Model

The collaborative layer runs on Yjs [7]. The shared canvas state lives in a `Y.Doc`, a Yjs shared document that can be replicated across any number of clients. Inside the document, a `Y.Map` named `shapes` stores all canvas shapes as serialised JSON values, keyed by their tldraw shape IDs.

When anything on the canvas changes, whether a shape is added, moved, resized, or deleted, the corresponding `Y.Map` entry is updated. Yjs encodes the change as a compact binary delta and sends it to the WebSocket provider, which broadcasts it to every other connected client. Each receiving client applies the delta to its local `Y.Doc` replica and merges the updated shapes into its local tldraw store via `editor.store.mergeRemoteChanges()`.

4.5.2 WebSocket Synchronisation Server

The Yjs WebSocket server is built on the `y-websocket` package, which handles WebSocket connection management and document broadcasting. In the deployed application it runs as part of the Express HTTP server on the same port, attaching to incoming WebSocket upgrade requests; for local development it can also be launched as a standalone Node.js process on port 1234. Either way, clients connect by specifying a room name in the WebSocket URL, and all clients in the same room share the same `Y.Doc`.

Documents stay in memory while at least one client is in their room. The standalone development server runs a cleanup task every 30 seconds that removes documents for empty rooms, preventing memory from growing indefinitely. When

a new client connects, it receives the full current document state from the server, allowing it to join an ongoing session and immediately see whatever work has already been done.

4.5.3 Awareness Protocol

Yjs includes an awareness protocol that lets clients broadcast ephemeral information: data that does not need to live in the shared document but that other clients should see in real time. Sketchly uses this to broadcast each user's cursor position and a display identifier. These are rendered as coloured overlays on the canvas, giving each user a live view of where their collaborators are working.

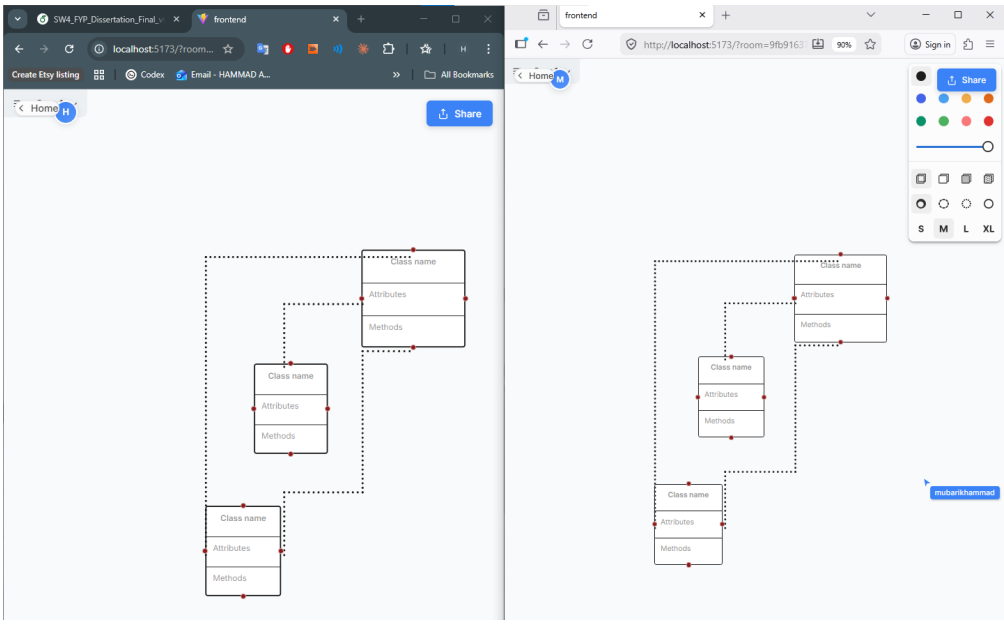

Figure 4.6: Two users collaborating in real time on a shared canvas.

4.6 AI-Powered Java Code Generation

4.6.1 Canvas Export

The Java code generation feature starts when the user clicks "Generate Java" in the toolbar. The frontend calls tldraw's `editor.getSvgString(shapeIds)` to get an SVG representation of all shapes on the current page. That SVG string is then drawn onto an off-screen HTML `canvas` element via an `Image` object, and the resulting bitmap is exported to a base64-encoded PNG string using

`canvas.toDataURL('image/png')`. This two-step SVG-to-PNG conversion avoids browser security restrictions that would arise from using a blob URL for the SVG source.

4.6.2 Backend Endpoint

The frontend sends a `POST` request to `/api/generate-java` on the Express backend, with the base64 PNG in the request body. The backend constructs a multimodal message to the Claude API with the image and a text prompt:

Listing 4.4: Claude API request for Java code generation.

```
{
  model: 'claude-opus-4-7',
  max_tokens: 4096,
  messages: [{
    role: 'user',
    content: [
      {
        type: 'image',
        source: { type: 'base64', media_type: 'image/png',
           data: image }
      },
      {
        type: 'text',
        text: 'This is a UML class diagram. Generate valid
           Java code
               for all classes shown. Use inheritance/
                  implementation
               relationships shown by the arrows. Return only
                   the
               Java code, no explanation.'
      }
    ]
  }]
}
```

The Claude Opus model was chosen for its strong vision-language reasoning, which produces accurate Java class structure from a rendered UML diagram. The prompt is intentionally minimal: a short instruction together with the diagram image, with no examples or constraints beyond "return only the Java code". Keeping the prompt small avoids biasing the model toward a particular template and lets it generalise across the variety of diagrams users actually draw.

4.6.3 Response Processing and Display

The API response contains the generated Java code as a text string. Before displaying it, the frontend applies a few cleaning steps:

- Markdown code fences (```` ```java ```` and ```` ``` ```) are stripped if present. The "return only the Java code, no explanation" instruction implicitly rules them out, but the model still sometimes wraps the response in fences.
- Common imports for Java types that the generated code references but does not declare (`java.util.List`, `java.util.ArrayList`, `java.util.Date`, and similar) are injected at the top of the file. Existing import statements are also hoisted to the top so the code compiles regardless of the order the model emitted them in.
- Class declarations missing a `public` modifier are updated to include one, since the model occasionally generates package-private classes.

The cleaned code appears in a modal with a Copy button that copies the code to the clipboard and a Save Files button that writes each class to a separate `.java` file, using the browser's File System Access API to save into a user-chosen folder. Figure 4.7 shows the code generation modal with an example output. Figure 4.8 shows the relationship between a drawn UML diagram and the resulting code.

Figure 4.7: The Java code generation modal showing generated class definitions.

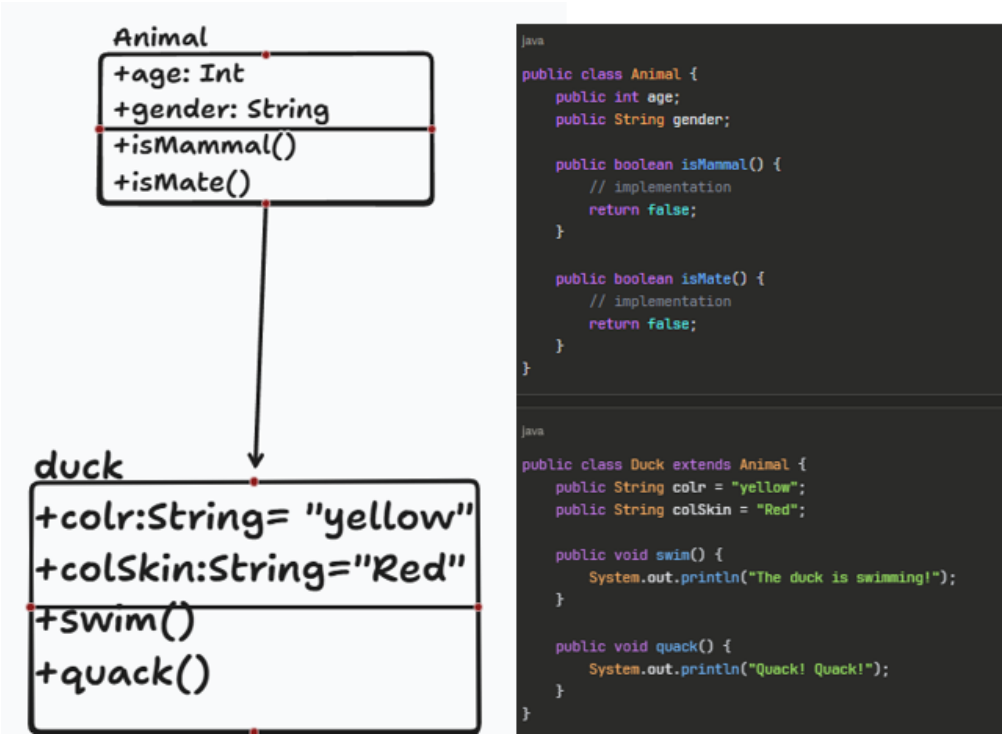

Figure 4.8: A hand-drawn UML class diagram (left) and the corresponding generated Java code (right).

4.7 Persistence and Authentication

4.7.1 User Authentication

Authentication is handled by Supabase Auth, which provides email and password login backed by a managed PostgreSQL user store. The frontend integrates through the `@supabase/supabase-js` client library. On login, the client receives a JWT used to authenticate subsequent database requests.

Unauthenticated users are redirected to a landing page. Access to the canvas requires a valid session. User identifiers are used to associate saved diagrams with their owners.

4.7.2 Diagram Persistence

Diagram state is saved to a `drawings` table in the Supabase PostgreSQL database. Each row stores a serialised `TLEditorSnapshot`, a JSON object holding the complete state of the tldraw canvas including all shapes, positions, properties, and the camera position. The `AutoSaveHandler` component subscribes to tldraw store change events and debounces writes to a maximum of once every five seconds, preventing a flood of database writes during active editing.

In collaborative rooms, the Yjs document is the primary source of truth for the current canvas state: when a new client joins a room, the Yjs WebSocket server replays the current document state to it, so the canvas is reconstructed from Y.js rather than from the Supabase snapshot. The Supabase snapshot serves as a durable backup of the diagram, written periodically by the auto-save handler so that the diagram's last known state survives a server restart or in-memory document loss.

Chapter 5

System Evaluation

Sketchly was built around five objectives. This chapter goes through each one, explains how it was tested, and presents the results. A brief discussion of overall system performance follows at the end.

5.1 Objective 1: Freehand Shape Recognition

5.1.1 Evaluation Method

Shape recognition was tested using a held out portion of the synthetic dataset produced by the same augmentation pipeline used for training. The full 100,000 images were split 90/10 with class-stratified sampling, leaving 1,000 images per class (10,000 in total) that were never seen by the model during training. Precision, recall, and F1 score were recorded for each of the ten classes, along with overall classification accuracy.

5.1.2 Results

The model reached an overall test accuracy of **99.28%**. Full per class results are in Table 5.1.

Shape Class	Precision	Recall	F1 Score	Support
circle	1.00	1.00	1.00	1,000
square	1.00	1.00	1.00	1,000
rectangle	1.00	1.00	1.00	1,000
triangle	0.98	0.97	0.97	1,000
arrow left	1.00	0.99	1.00	1,000
arrow right	1.00	0.99	1.00	1,000
arrow up	1.00	1.00	1.00	1,000
arrow down	1.00	0.99	1.00	1,000
diamond	0.99	1.00	0.99	1,000
line	0.98	0.99	0.98	1,000
Overall accuracy		**99.28%**		
Macro average	0.99	0.99	0.99	10,000

Table 5.1: Per class precision, recall, and F1 score on the held out test set.

5.1.3 Analysis

Eight of the ten shape classes score F1 = 1.00 or 0.99. The weakest result is for triangle at 0.97, which is expected: a narrow triangle and a short and wide rectangle can look very similar in a 48×48 rasterised image, especially when drawn on a screen with a mouse. The bounding lines blur and the three sided structure becomes hard to distinguish from four sides at that resolution. The geometric post processing stage catches these borderline cases. When the CNN returns a low confidence rectangle prediction, the corner counting step checks whether the stroke has three corners rather than four, and if so reclassifies to triangle. This correction is not visible in the table above, which measures the CNN alone. In practical use, drawing real triangles on the canvas, the hybrid system gets the right answer consistently.

All four arrow classes score perfectly. Arrows have a strong directional signature with a clear asymmetry between head and tail that the other shape classes do not share, so there is very little chance of confusion between them or with other classes.

5.1.4 Comparison with the Original Geometric Recogniser

The geometric recogniser that preceded the CNN was not formally benchmarked, but its failure modes were clear from testing. Triangles came out as rectangles regularly. Squares and rectangles were frequently confused when the drawing was not perfectly axis aligned. Shapes drawn quickly at small scales were misclassified

often enough that using it as the sole recogniser was not viable. The CNN solves all of these problems while adding only 5 to 20 milliseconds of inference time per stroke, which is well within the range where it feels instant to the user.

5.2 Objective 2: Real Time Collaborative Editing

5.2.1 Evaluation Method

Collaborative editing was tested functionally rather than quantitatively. Two browser tabs were opened on the same room simultaneously. Shapes were added, moved, resized, and deleted rapidly in both tabs, including cases where both tabs modified the same shape at the same time. After each batch of edits, the canvas state in both tabs was inspected visually.

The four things checked were:

1. Changes made in one tab appeared in the other within 100ms.
2. Nothing was lost: every shape added in one tab was present in the other after synchronisation.
3. Nothing was duplicated: a shape created once appeared exactly once on both canvases.
4. Concurrent edits to the same shape (moving it in one tab while resizing it in the other) resolved to a consistent final state in both tabs.

5.2.2 Results

All four checks passed on every test run. Concurrent edits propagated reliably and both clients always ended up with the same canvas state. The CRDT merge properties of Yjs [7, 3] handled every concurrency scenario encountered without any extra conflict resolution code in the application layer.

Cursor positions, transmitted via the Yjs awareness protocol, updated in real time and showed up correctly as coloured overlays on both canvases. No stale or frozen cursor positions were observed.

5.2.3 Limitations

Testing was done with both tabs connected to a locally running WebSocket server. Network round trip times in a real deployment will be higher, especially for geographically distributed users. The observed sub 100ms synchronisation latency is therefore a best case figure, not a guarantee for production.

The WebSocket server holds document state in memory only. If the server process restarts while users are connected, the in memory state is lost and clients need to reconnect. The Supabase auto save mechanism reduces this risk for saved diagrams, but any edits made since the last auto save interval (five seconds) would be lost in a server crash.

5.3 Objective 3: Intelligent Connector Routing

5.3.1 Evaluation Method

Connector routing was evaluated visually by building diagrams with a range of shape arrangements: shapes in rows, shapes in columns, and tightly packed layouts with very little clear space. The main thing checked was whether any routed connector crossed through the body of a shape that was not its source or target. Secondary checks were whether paths were unreasonably long and whether routing completed without any visible delay.

5.3.2 Results

No routed connector was observed passing through another shape in any test configuration. Even in tightly constrained layouts where there was only a single narrow corridor between shapes, the A* search [2] found it and threaded the connector through. Routing completed in under 5ms in all cases, which is fast enough that the connector appears on screen the moment the user releases the pointer.

The 25 pixel grid resolution produced paths that are smooth and visually clean at typical diagram scales. Paths are orthogonal, which matches the expected UML and software architecture diagram style.

5.3.3 Limitations

The fixed 25 pixel grid resolution means the router cannot find a path through a gap narrower than 25 pixels. If two shapes are placed very close together or overlapping, the router falls back to a simple L-shaped orthogonal path rather than failing silently. In practice this is rarely a problem, since users do not intentionally place shapes less than 25 pixels apart.

To keep paths visually clean, the A* search adds a per-turn cost (the bend penalty) to each step that changes direction, and an additional penalty for arriving at the end cell from the wrong direction. This makes the search prefer routes with fewer corners and the correct entry/exit direction at connection points, rather

than choosing arbitrarily between equally short paths. A more aggressive post processing simplification pass could still improve aesthetics in tightly packed layouts and was noted as a future improvement.

5.4 Objective 4: UML Class Diagram Support

5.4.1 Evaluation Method

The UML class shape was tested across several scenarios: entering text in all three compartments, watching the shape auto resize as text was added, pasting in large blocks of text, editing text in the middle of an existing string to check cursor preservation, and connecting two class shapes with a routed connector, then moving one shape to confirm the connector stayed attached and was rerouted.

5.4.2 Results

Every scenario behaved correctly. Auto resize updated the shape height to fit the content without visible flicker and without triggering infinite render loops. Pasting multiple lines of text into a compartment caused the shape to expand immediately. Cursor position was preserved correctly after every keystroke and after every paste in all three compartments, staying at the position where the edit was made rather than jumping to the end.

Connectors stayed attached when shapes were moved and the connector path was recomputed automatically on each move event.

5.5 Objective 5: AI Powered Java Code Generation

5.5.1 Evaluation Method

Java code generation was tested by drawing two UML class diagrams by hand on the canvas and generating code from each. The output was inspected manually against the diagram to check whether all expected elements were correctly reproduced. The two diagrams were:

1. An **Animal/Duck** hierarchy: an `Animal` abstract class with fields `age` and `gender` and a method `isDomesticated()`, and a `Duck` class extending it with fields `colourString` and `colourSkin` and methods `swim()` and `quack()`.

2. A **Document/Book** hierarchy: a `Document` abstract class with a `String[]` field `authors` and a `Date` field `date`, and a `Book` class extending it with a `String` field `title` and a method `addAuthors(String name)`.

5.5.2 Results

Both diagrams produced correct Java. Every expected element was present and correct in the output. Table 5.2 shows the breakdown.

Diagram	Expected element	Correct?
Animal/Duck	`abstract class Animal`	✓
	`class Duck extends Animal`	✓
	Field names and types (`age`, `gender`, etc.)	✓
	Method signatures (`isDomesticated()`, `swim()`, `quack()`)	✓
	Getter/setter methods generated for fields	✓
Document/Book	`abstract class Document`	✓
	`class Book extends Document`	✓
	`String[] authors` field	✓
	`Date date` field	✓
	`String title` field in Book	✓
	`addAuthors(String name)` method	✓

Table 5.2: Assessment of Java code generation correctness for two test diagrams.

The generated code compiled without errors when copied into a Java development environment.

5.5.3 Limitations

Code generation quality depends directly on the quality of the diagram. The Claude model [12] reads the diagram as an image, so overlapping shapes, labels that are too small, or ambiguous arrow directions can all cause the output to be wrong or incomplete. A clearly drawn diagram produces reliable results; a messy one may not.

The feature also depends on the Anthropic API being available. If the API is down, code generation is unavailable entirely. There is also a per request API cost and a network round trip that takes one to three seconds, which users will notice. This latency is fine for an operation that happens occasionally, but would not be acceptable for something triggered frequently.

Method bodies in the generated code are typically stubs rather than full implementations, which is correct given that a UML class diagram shows signatures, not logic.

5.6 Performance Characteristics

Table 5.3 summarises the observed latency and timing characteristics of the key system components.

Component	**Observed performance**
CNN model loading	50 200ms on first page load (weights served as static asset)
CNN inference (per stroke)	5 20ms including rasterisation
Geometric correction	<1ms
Collaborative sync latency	<100ms on local network
Auto save debounce	5 seconds
Java code generation	1 3 seconds (network round trip and API processing)

Table 5.3: Observed performance characteristics of key system components.

The only component with a latency that users will notice is the Java code generation pipeline. A one to three second wait is acceptable for a one off export operation, but it would be unacceptable for something that runs on every interaction. Every other component runs within a few milliseconds: shape recognition completes before the user's hand has moved away from the canvas, and collaborative changes appear on the remote client in under a second even on a local network.

Chapter 6

Conclusion

6.1 Summary

Sketchly started from a simple observation: existing diagramming tools make you choose between speed and structure. You can sketch an idea in seconds on Excalidraw, but what you get is a picture, not data. The aim of this project was to build something that does both at once.

The system that resulted combines freehand stroke input with a CNN-based shape classifier [16], real-time multi-user collaboration via CRDTs [3], orthogonal connector routing with A* pathfinding [2], a custom UML class diagram shape type, and a one-click Java code generator. All of this runs in the browser, with no server round trip required for shape recognition.

The five objectives set out at the start of the project are reviewed below.

6.1.1 Objective 1: Freehand Shape Recognition

The shape recognition pipeline was implemented as a hybrid of a CNN [17] and a geometric corner-counting correction stage. The CNN was trained on 100,000 synthetic images across ten shape classes, produced by an augmentation pipeline that simulates hand tremor, elastic deformation, variable stroke speed, and partial shape incompleteness.

The trained model reached 99.28% overall accuracy on the held-out test set. F1 scores were 1.00 for circles, squares, rectangles, and all four arrow classes, 0.99 for diamonds, 0.98 for lines, and 0.97 for triangles. The triangle result reflects the genuine visual ambiguity between a flat triangle and a short rectangle at 48×48 resolution. The geometric correction stage reduces real-world errors for this class by checking the corner count when the CNN is uncertain.

This objective was met.

6.1.2 Objective 2: Real-Time Collaborative Editing

Collaborative editing was built on Yjs [7] CRDTs [3] connected over a Node.js WebSocket server. Testing with concurrent edits from multiple browser tabs showed consistent propagation, no lost edits, no duplicated shapes, and identical canvas states on all clients after synchronisation. The CRDT merge semantics handled every concurrency scenario without any application-level conflict resolution code.

This objective was met.

6.1.3 Objective 3: Intelligent Connector Routing

Connector routing uses A* [2] on a 25-pixel grid with shape bounding boxes marked as impassable. In all test layouts, including dense configurations with very little free space, routed connectors followed orthogonal paths that stayed outside non-endpoint shapes. Routing completed in under 5ms for every tested diagram.

The main limitation is the fixed grid resolution, which means paths cannot thread through gaps narrower than 25 pixels. Bend minimisation is handled inside the search itself via a per-turn cost in A*, and a more aggressive post processing simplification pass remains as future work. These are reasonable trade offs for a prototype.

This objective was substantially met.

6.1.4 Objective 4: UML Class Diagram Support

The UML class shape was implemented as a custom tldraw [6] shape type with three editable text compartments, auto-resize based on measured scroll height, cursor preservation via `useLayoutEffect`, and four connection points. All tested scenarios worked correctly: multi-line text, large paste operations, mid-string edits, and collaborative editing by multiple users on the same shape.

Full UML notation support [4] (stereotypes, visibility modifiers, all relationship arrowhead styles) was out of scope and remains as future work.

This objective was met within the defined scope.

6.1.5 Objective 5: AI-Powered Java Code Generation

Java generation was implemented by rendering the canvas to a PNG and sending it to Claude Opus [12] with a prompt requesting Java class definitions from the UML structure. Testing on two hand-drawn diagrams (Animal/Duck and Document/Book) produced correct output in both cases: class names, inheritance relationships, field names, field types, and method signatures were all reproduced accurately. The generated code compiled without errors.

Output quality depends on the visual clarity of the input. Overlapping shapes or illegible labels will produce less reliable results, which is an inherent limitation of the vision-based approach.

This objective was met for well-drawn, unambiguous diagrams.

6.2 Reflections on Technical Decisions

6.2.1 Synthetic Data Augmentation

Generating the training data synthetically rather than collecting it from real users was a practical decision made under time pressure, but it turned out to be fine. The augmentation pipeline was critical to making the model work on real free-hand input. Without tremor simulation and elastic deformation, training on clean geometric shapes would have produced a model that failed on the noisy, variable strokes that real users draw. The result, 99.28% on a synthetic test set backed by correct classifications on real strokes, suggests the pipeline approximated real variation well enough.

6.2.2 Hybrid CNN and Geometric Correction

Separating the CNN from the geometric correction stage proved more robust than trying to get either to handle everything alone. The CNN [16] is accurate and fast for the vast majority of inputs. The corner counting correction exists to handle one specific case where the CNN is structurally weak: distinguishing a triangle from a low confidence rectangle when both are drawn with similar bounding boxes. Because the correction runs as a post-processing step, it can be tuned independently without affecting the model, and failures in one component do not automatically pull the other down with them.

6.2.3 CRDTs over Operational Transformation

Choosing Yjs [7] over a custom OT-based sync layer saved a significant amount of implementation work. A basic collaborative session was working within hours of integrating the Yjs tldraw binding. OT would have required a centralised server to impose a total operation order [19], adding both infrastructure complexity and a single point of failure. The CRDT approach [3] means the system could in principle support peer-to-peer sessions with no central server beyond the initial signalling step [20].

6.2.4 In-Browser CNN Inference

Running inference via TensorFlow.js [14] rather than sending strokes to a backend server had a noticeable effect on the experience. The 5 to 20ms inference time means shape recognition feels instant. A server based approach would have added network latency on every stroke, which would make the freehand interaction feel sluggish. It also means recognition still works when the user has no network connection, and no stroke data is sent to an external service.

6.3 Limitations and Future Work

6.3.1 Formal User Study

The evaluation did not include a structured study with real participants. The 99.28% accuracy figure comes from a synthetic test set, and while informal testing on real strokes looked good, that is not the same as measuring task completion time, error rates, and user satisfaction across a representative group. A future study could compare Sketchly against draw.io on a set diagramming task, measuring the time taken and the proportion of correctly placed and labelled shapes.

6.3.2 Extended UML Support

The system currently covers the class diagram subset of UML [4]. Sequence diagrams and activity diagrams are the next most commonly used types in software engineering practice [1], but they need a different visual language. Sequence diagrams use lifelines, activation boxes, and message arrows that do not map to the current shape types. Adding them would require new recognition classes, new shape types in tldraw [6], and extended prompt engineering in the code generation step.

6.3.3 Fine-Tuning on Real Hand-Drawn Data

Training on real strokes drawn through the application, with user consent, would likely improve robustness on the specific drawing patterns that appear in practice. The synthetic augmentation pipeline approximates real variation but cannot replicate it exactly. Even a small dataset of 500 to 1000 real examples per class would be enough for fine tuning without requiring a large collection effort.

6.3.4 Offline and Progressive Web App

Shape recognition and canvas editing work offline today, since the model weights are served as static assets. But changes made while offline are not saved unless a server connection is available. Implementing a progressive web app with a service worker and an IndexedDB backend would allow full offline operation, with changes syncing to the server when connectivity returns.

6.3.5 Multi-Language Code Generation

The code generation prompt is written to request Java specifically. Extending this to Python, C#, or TypeScript would require only a prompt change and minimal backend work. The vision-language model [12] already understands the semantics of those languages, so the main effort would be ensuring that the output is formatted and post-processed correctly for each target language.

6.4 Conclusion

Sketchly shows that combining freehand input, shape recognition, orthogonal routing, real-time collaboration, and AI code generation in a single browser-based application is feasible using current tooling. TensorFlow.js [14], Yjs [7], tldraw [6], and the Claude API [12] are all mature enough that integrating them did not require solving deep technical problems from scratch. The contribution of this project was making them work together in a way that produces a coherent user experience.

The most important lesson from building it was the value of iterative development when the technical ground is uncertain. The switch from the geometric recogniser to the CNN, the tuning of the geometric correction thresholds, and the fix for the infinite render loop introduced by a naive `useLayoutEffect` dependency array were all discovered through implementation and testing, not upfront design. A plan that tried to specify all of these decisions before writing any code would have got most of them wrong.

Bibliography

[1] Andrew Forward and Timothy C Lethbridge. The relevance of software documentation, tools and technologies: A survey. In *Proceedings of the 2002 ACM Symposium on Document Engineering*, pages 26–33. ACM, 2002.

[2] Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[3] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, volume 6976 of *Lecture Notes in Computer Science*, pages 386–400. Springer, 2011.

[4] Object Management Group. UML 2.5.1 specification. `https://www.omg.org/spec/UML/2.5.1`, 2017.

[5] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, et al. Manifesto for agile software development. `https://agilemanifesto.org`, 2001.

[6] tldraw contributors. tldraw. `https://tldraw.dev`, 2023.

[7] Petru Nicolaescu, Kevin Jahns, Michael Derntl, and Ralf Klamma. Near real-time peer-to-peer shared editing on extensible data types. In *Proceedings of the 19th International Conference on Supporting Group Work*, pages 39–49. ACM, 2016.

[8] Jacob O Wobbrock, Andrew D Wilson, and Yang Li. Gestures without libraries, toolkits or training: a $1 recognizer for user interface prototypes. In *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology*, pages 159–168. ACM, 2007.

[9] Jonas Jongejan, Henry Rowley, Takashi Kawashima, Jongmin Kim, and Nick Fox-Gieg. The quick, draw! dataset. `https://github.com/googlecreativelab/quickdraw-dataset`, 2016.

[10] Clarence A Ellis and Simon J Gibbs. Concurrency control in groupware systems. In *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, pages 399–407, 1989.

[11] David A Nichols, Pavel Curtis, Michael Dixon, and John Lamping. High-latency, low-bandwidth windowing in the Jupiter collaboration system. In *Proceedings of the 8th Annual ACM Symposium on User Interface Software and Technology*, pages 111–120. ACM, 1995.

[12] Anthropic. Claude Opus 4.7. `https://www.anthropic.com/claude`, 2026.

[13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[14] Daniel Smilkov, Nikhil Thorat, Yannick Assogba, Ann Yuan, Nick Kreeger, Ping Yu, Kangyi Zhang, Shanqing Cai, Eric Nielsen, David Soergel, et al. TensorFlow.js: Machine learning for the web and beyond. *Proceedings of Machine Learning and Systems*, 1:309–321, 2019.

[15] Dean Rubine. Specifying gestures by example. In *Proceedings of the 18th Annual Conference on Computer Graphics and Interactive Techniques*, pages 329–337. ACM, 1991.

[16] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 2012.

[17] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[18] David Ha and Douglas Eck. A neural representation of sketch drawings. *arXiv preprint arXiv:1704.03477*, 2017.

[19] Matthias Ressel, Doris Nitsche-Ruhland, and Rul Gunzenhäuser. An integrating, transformation-oriented approach to concurrency control and undo in group editors. In *Proceedings of the 1996 ACM Conference on Computer Supported Cooperative Work*, pages 288–297, 1996.

[20] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, pages 154–178. ACM, 2019.

[21] Michael R Garey and David S Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[22] Tomás Lozano-Pérez and Michael A Wesley. An algorithm for planning collision-free paths among polyhedral obstacles. *Communications of the ACM*, 22(10):560–570, 1979.

[23] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International Conference on Machine Learning*, pages 448–456, 2015.

[24] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. In *International Conference on Learning Representations*, 2014.

[25] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

Appendix A

Source Code Repository

The complete source code for Sketchly, including the React/TypeScript frontend, the Express/Yjs backend, the Python training pipeline, and this dissertation's LaTeX sources, is hosted on GitHub at:

`https://github.com/HammadMubarik/sketchly`

The repository contains:

- `frontend/` — React + TypeScript single-page application built with Vite, including the tldraw canvas integration, the CNN recogniser ([frontend/src/lib/cnnRecognizer.ts]), the A* pathfinder ([frontend/src/lib/pathfinder.ts]), the UML class shape util, and the Yjs collaboration bridge.
- `backend/` — Express.js server that hosts the Java code generation REST endpoint and accepts WebSocket upgrades for the Yjs synchronisation layer.
- `training/` — Python training pipeline ([training/train_model.py]) that generates the synthetic dataset, trains the CNN, and exports it to TensorFlow.js format.
- `Dockerfile` — Multi-stage build that bundles the frontend and backend into a single Node process for deployment.

Setup instructions for running the system locally are documented in the repository's `README`.